Atividade Lab - Processamento Distribuído de Logs com MPI

MPI + Python + Docker

Prof Alcides/Prof Mário

1. Contexto e Objetivos

Em sistemas reais de grande porte, como plataformas de streaming, lojas virtuais, redes sociais, bancos digitais e serviços em nuvem, os servidores registram continuamente informações sobre as operações do sistema. Esses registros, chamados de logs, guardam informações como a máquina requisitante, o recurso acessado e o status da requisição (sucesso ou erro).

Esses logs são fundamentais para o monitoramento, identificação de falhas, análise de desempenho e auditoria. No entanto, em ambientes reais, a quantidade de logs pode ser extremamente grande, atingindo milhões de linhas em pouco tempo. Processar esse volume em um único processo torna-se lento e pouco escalável, cenário ideal para a aplicação da computação distribuída.

O objetivo desta atividade é desenvolver um programa em Python com a biblioteca `mpi4py`, utilizando o método de comunicação Scatter, para dividir um grande conjunto de logs entre diferentes processos. O foco é a compreensão do particionamento e distribuição de dados; cada processo enviará ao master a sua contagem em uma única string já formatada por mensagem ponto a ponto; não é necessário consolidar o resultado final em um único processo utilizando Reduce ou Gather.

2. Especificação Técnica do Problema

O programa distribuído deve ser estruturado nas cinco etapas descritas a seguir:

Etapa 1: Geração do Dataset

O processo principal (rank 0) deverá gerar localmente um conjunto grande de logs simulados (ex: 500.000 a 1.000.000 de linhas). Cada linha deve seguir o padrão: `IP MÉTODO ENDPOINT STATUS`.

Exemplos:

```
192.168.1.10 GET /login 200
192.168.1.23 GET /checkout 500
```

Os códigos de status a serem considerados para a análise são: `200` (sucesso), `404` (não encontrado) e `500` (erro interno).

Etapa 2: Divisão dos Dados

O processo 0 deverá dividir o conjunto de logs gerado em partes iguais, correspondentes à quantidade de processos MPI instanciados. Se houver 4 processos e 100.000 linhas, cada processo deverá receber exatamente 25.000 linhas.

Etapa 3: Distribuição com Scatter

A distribuição das partes para os demais nós deverá ser feita obrigatoriamente utilizando a rotina de comunicação coletiva `comm.scatter(...)`. Após esta operação, cada processo terá em memória apenas a sua fração do dataset original.

Etapa 4: Processamento Local

Cada processo analisará estritamente as linhas que recebeu. O algoritmo deve separar os campos da string de log, identificar o código de status e contabilizar a quantidade de erros (status `404` ou `500`) presentes em sua partição.

Etapa 5: Saída de Cada Processo

Cada processo deverá enviar uma mensagem para o master informando o seu rank, o volume de linhas recebidas e a quantidade de erros detectados.

Exemplo de saída esperada no processo Master:

Processo 0 analisou 25000 linhas e encontrou 8241 erros.
Processo 1 analisou 25000 linhas e encontrou 8175 erros.

3. Requisitos Obrigatórios

A implementação final deve cumprir as seguintes restrições:

1. Ser implementada em Python.
2. Utilizar a biblioteca `mpi4py`.
3. O dataset deve ser gerado em memória pelo processo 0, sem a necessidade de leitura de arquivos externos.
4. A distribuição dos dados deve utilizar exclusivamente a rotina Scatter.
5. Cada nó processa apenas sua fatia de dados.
6. Cada processo envia para o master o número de linhas processadas e os erros encontrados; o master vai imprimir no seu terminal, como mostrado acima.

4. Template de Código Inicial

Abaixo encontra-se a estrutura base para iniciar o desenvolvimento da solução. Substitua os comentários em maiúsculo pela lógica correspondente.

```python
from mpi4py import MPI
import random

# Inicialização do MPI
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

#
# Função para gerar logs
#
def gerar_logs(qtd):
    ips = [f"192.168.1.{i}" for i in range(2,254)]
    endpoints = [
        "/",
        "/login",
        "/products",
        "/cart",
        "/checkout",
        "/api/users",
        "/api/orders"
    ]
    metodos = ["GET", "POST"]
    status = ["200", "200", "200", "404", "500"]
```

```
    logs = []
    # INSIRA AQUI O SEU CÓDIGO PARA GERAR OS DADOS DO LOG
    # RANDOMIZE OS IPS, ENDPOINTS, METODOS E STATUS

    return logs

#
# Processo 0 gera o dataset
#
logs_divididos = None
TOTAL_LOGS = 100000

if rank == 0:
    print("\nGerando dataset de logs...\n")
    logs = gerar_logs(TOTAL_LOGS)

    # DIVIDIR AQUI O DATASET ENTRE OS PROCESSOS
    # O NÓ MASTER TAMBÉM PROCESSA SUA PARTE DO DATASET

#
# Distribuição usando Scatter
#
# DISTRIBUIR OS DADOS USANDO SCATTER
#


#
# Processamento local em cada nó
#
# INSIRA AQUI O CÓDIGO DE PROCESSAMENTO LOCAL DO LOG
#

# Nós Master
# Imprime os resultados de cada nó worker e seu também
#
# A variável 'logs_locais' representa a fatia recebida e 'erros' o contador
print(f"Processo {rank} analisou {len(logs_locais)} linhas "
      f"e encontrou {erros} erros."
)
```